Практическая работа №1

Пользовательские запросы в веб-приложении (DevTools, HTTP, REST, JSON, Postman)

1.Теория:

1.1. Что такое DevTools

DevTools (Developer Tools) – встроенные инструменты разработчика браузера (Chrome, Firefox, Edge, Safari).

Открытие: F12.

Основные панели:

– Elements – DOM и визуальная структура страницы.

– Console – ошибки, логи, выполнение JS.

– Network – анализ запросов, коды, заголовки, время.

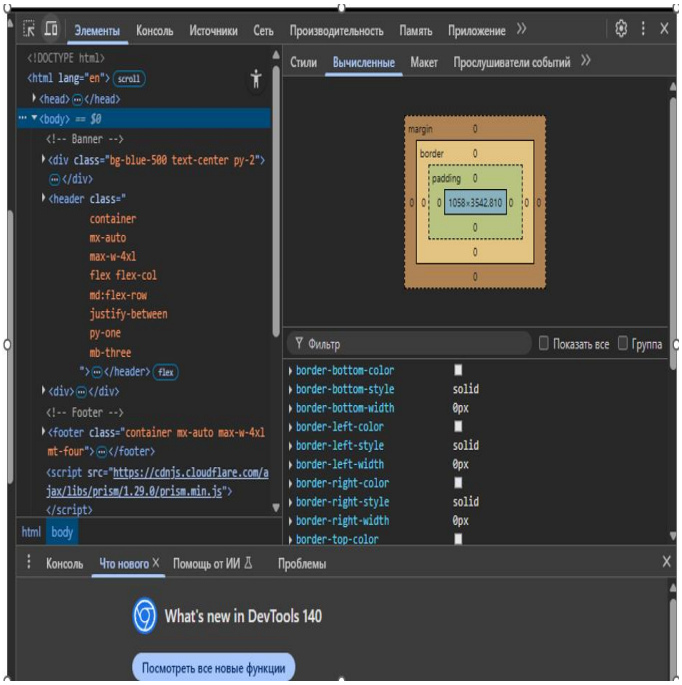
– Performance – измерение скорости работы страницы.

– Device mode – проверка адаптивности интерфейса.

– Security – HTTPS, сертификаты.

– Memory – утечки памяти и использование ресурсов.

Пример интерфейса DevTools:



1.2. Пользовательские запросы

Пользовательские запросы – это HTTP-запросы (GET, POST и др.), которые браузер отправляет при работе с веб-приложением.

Основные характеристики запроса:

– метод (GET/POST/PUT/DELETE/…)

– URL и параметры

– заголовки (headers)

– тело запроса / ответа

– время выполнения

– код ответа

Оптимизация запросов:

– уменьшение количества обращений;

– сокращение объёма передаваемых данных;

– кэширование;

– пагинация и фильтрация;

– использование правильных HTTP-методов и заголовков.

Основные HTTP-методы

Метод	CRUD	Назначение	Особенности	Пример
GET	Read	Получение данных	Параметры в URL, кэшируемый, не меняет данные	/users?id=5
POST	Create	Создание ресурса	Данные в теле, изменяет состояние	Регистрация, заказ
PUT	Update	Полное обновление	Заменяет весь объект	Обновить профиль
PATCH	Update	Частичное обновление	Меняет только указанные поля	Сменить только email

DELETE	Delete	Удаление ресурса	Обычно 200 OK или 204 No Content	Удалить товар
HEAD	–	Только заголовки	Проверка наличия / размера / даты	Проверка Last-Modified

1.3. Коды ответов

– 1xx – информационные (например, 100 Continue).

– 2xx – успешные (200 OK, 201 Created, 204 No Content).

– 3xx – перенаправления (301, 302, 304 Not Modified).

– 4xx – ошибки клиента (400, 401, 403, 404).

– 5xx – ошибки сервера (500, 502, 503, 504).

1.4. REST API

REST – архитектурный стиль взаимодействия с ресурсами по HTTP.

Принципы:

– клиент–сервер;

– отсутствие состояния (stateless);

– единообразный интерфейс;

– работа с ресурсами (уникальные URL);

– часто используется формат JSON.

1.5. JSON

JSON – лёгкий текстовый формат обмена данными (объекты, массивы, строки, числа, boolean, null).

Пример ответа:

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere...",
  "body": "quia et suscipit..."
}
```

1.6. Postman

Postman – инструмент для работы с API:

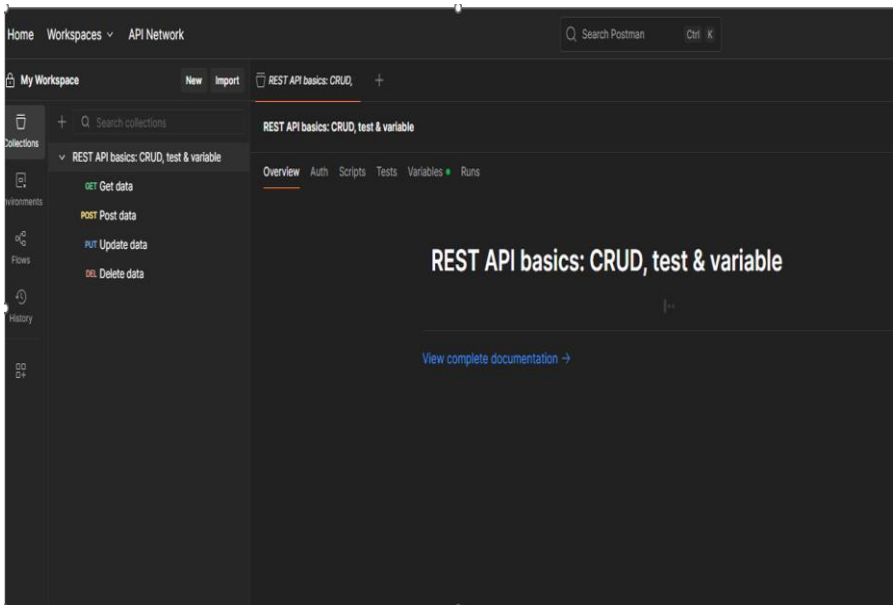
– создание и отправка запросов;

– анализ ответов;

– сохранение коллекций;

– автотесты;

– совместная работа.

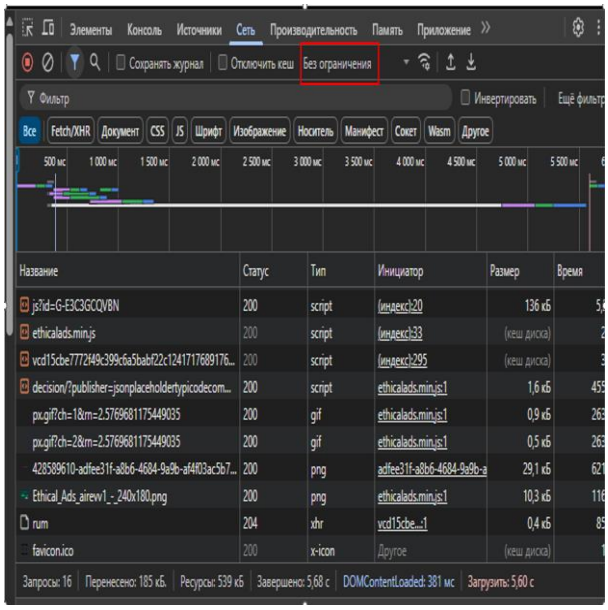


2. Задание

2.1. Требуемые инструменты

– браузер (Chrome / Firefox / Edge / Яндекс.Браузер);

– DevTools вкладка Network (режим Online, без ограничения скорости или с 4G-профилем).



2.2. Первичный анализ запросов

1. Откройте DevTools (F12) вкладка Network.

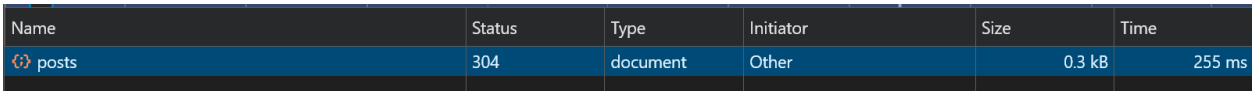
2. Загрузите страницу:

https://jsonplaceholder.typicode.com/posts

1. Зафиксируйте в таблице: метод, URL, код ответа, размер данных, время загрузки.

GET	https://jsonplaceholder.typicode.com/posts	304	0.3 КБ	255ms

2. Сделайте скриншот списка запросов.



2.3. Детальный разбор одного запроса

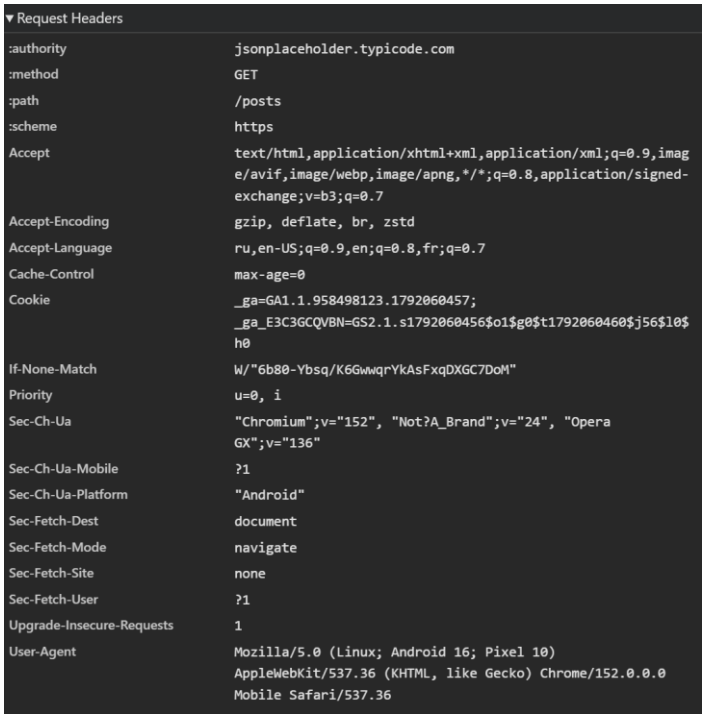
Запрос GET https://jsonplaceholder.typicode.com/posts:

– Метод - GET;

– URL и параметры - https://jsonplaceholder.typicode.com/posts, без параметров;

– заголовки (Request Headers / Response Headers);

Request Headers:



▾ Request Headers

:authority	jsonplaceholder.typicode.com
:method	GET
:path	/posts
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	ru,en-US;q=0.9,en;q=0.8,fr;q=0.7
Cache-Control	max-age=0
Cookie	_ga=GA1.1.958498123.1792060457; _ga_E3C3GCQVBN=GS2.1.s1792060456$o1$g0$t1792060460$j56$l0$h0
If-None-Match	W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"
Priority	u=0, i
Sec-Ch-Ua	"Chromium";v="152", "Not?A_Brand";v="24", "Opera GX";v="136"
Sec-Ch-Ua-Mobile	?1
Sec-Ch-Ua-Platform	"Android"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Linux; Android 16; Pixel 10) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Mobile Safari/537.36

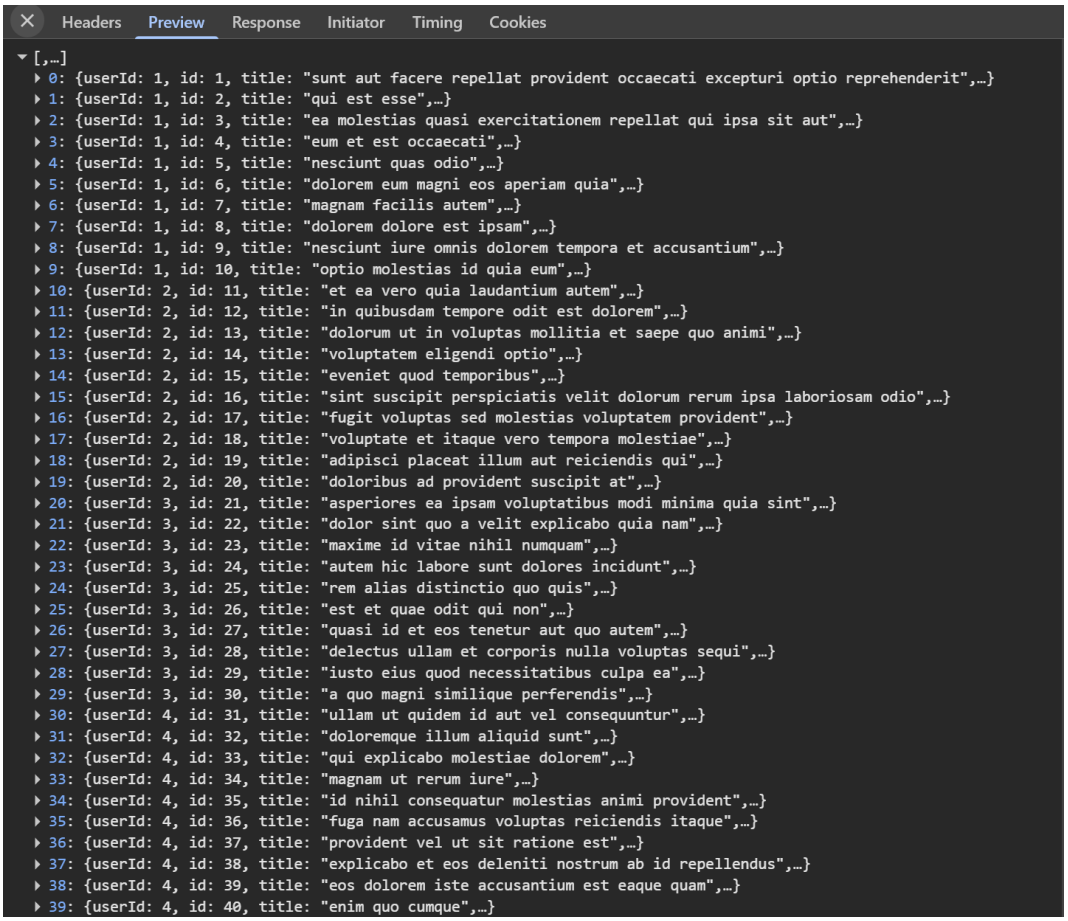
Response Headers:



▾ Response headers

Access-Control-Allow-Credentials	true
Age	26633
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	a4ae3059cae4dda0-HKG
Date	Thu, 15 Oct 2026 10:37:30 GMT
Etag	W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"
Expires	-1
Nel	{"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Pragma	no-cache
Report-To	{"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=5EhNWaIu3gOvap4doL%2BsYALKLCdc0igcYtKB7oxSpT2d05QpuU0G%2BRlduvPiP6nGUZcMfreeoc0roh%2BjKVdNkKaTS%2Bz7Q0rhCcnFqMCSHPnAYvmN5CLSGLYYjEJSOZMmQICTTOhZS90Nl%2FsPRawI"}]}
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdge;dur=5,cfOrigin;dur=0
Vary	Origin, Accept-Encoding
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	3600000
X-Ratelimit-Remaining	3599999
X-Ratelimit-Reset	1790812826

– тело ответа (Response / Preview):



– код ответа – 304;

– время выполнения – 240ms.

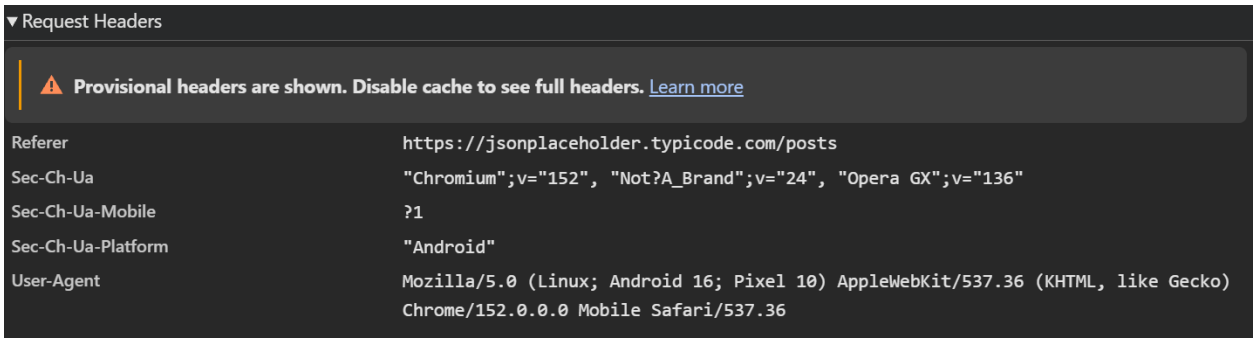
Сравнение с запросом GET https://jsonplaceholder.typicode.com/favicon.ico.

– Метод - GET;

– URL и параметры - https://jsonplaceholder.typicode.com/ favicon.ico, без параметров;

– заголовки (Request Headers / Response Headers);

Request Headers:

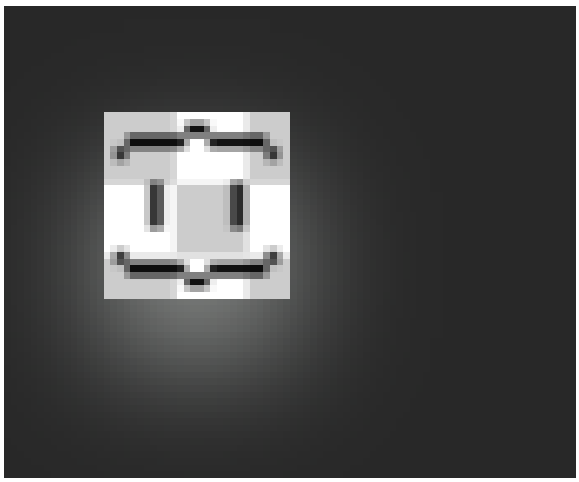


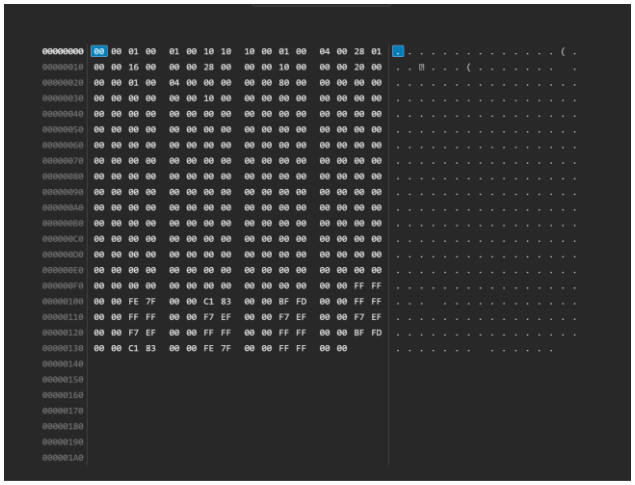
Response Headers:

▾ Response headers

Access-Control-Allow-Credentials	true
Age	12443
Alt-Svc	h3=":443"; ma=86400
Cache-Control	public, max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	a4ae2b9afe89dda0-HKG
Content-Encoding	zstd
Content-Type	image/x-icon
Date	Thu, 15 Oct 2026 10:34:16 GMT
Etag	W/"13e-1a12fee1ea8"
Last-Modified	Mon, 12 Oct 2026 11:44:57 GMT
Nel	{"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Report-To	{"group":"cf-nel","max_age":604800,"endpoints": [{"url":"https://a.nel.cloudflare.com/report/v4?s=k0huo5BVl8omxV4%2BBkTbJwdb1eoZMx%2FVPA%2BJBwE%2BtSlTUG%2FrWYC%2BN9l7TliIFWc2IKqmfm8%2BpU9934Kx6wEEuziW7iGmEp7P6bgfjVej%2F%2F7TAKdrlOalPIfkuavlbNCpczlUaL1NqDEZ30VhFte4"}]}
Server	cloudflare
Vary	Origin, Accept-Encoding
X-Powered-By	Express
X-Ratelimit-Limit	3600000
X-Ratelimit-Remaining	3599999
X-Ratelimit-Reset	1791820195

– тело ответа (Response / Preview):





– код ответа – 200;

– время выполнения – 1ms.

2.4. Эксперименты с запросами

Выполните в консоли:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(res => res.json())
```

```
  .then(data => console.log(data));
```

Проанализируйте запрос во вкладке Network. Сравните:

– GET /posts (все посты);

– GET /posts/1 (один пост).

Сделайте выводы по:

– размеру ответа:

GET /posts/ – 0.3KB

GET /posts/1 – 0.6KB

– времени загрузки;

GET /posts/ – 240ms

GET /posts/1 – 228ms

– количеству записей.

GET /posts/ – 100

GET /posts/1 – 1

ПОЧЕМУ-ТО ЗАПРОС НА 100 ЗАПИСЕЙ ВЕСИТ МЕНЬШЕ И ОБРАБАТЫВАЕТСЯ СТОЛЬКО ЖЕ, А ТО И БЫСТРЕЕ ЧЕМ ЗАПРОС НА ОДНУ ЗАПИСЬ. ВОЗМОЖНО ДЕЛО В ПАРАМЕТРАХ. ХОТЯ ЕСЛИ ПОВТОРНО ОТПРАВИТЬ ОБА ЗАПРОС ДЕЛО ВЫРАВНИВАЕТСЯ, НО ВСЁ РАВНО СТРАННО

2.5. Работа с параметрами

1. Откройте в браузере:

```
https://jsonplaceholder.typicode.com/posts?userId=1
```

1. Сравните с https://jsonplaceholder.typicode.com/posts:

2. сколько объектов вернулось – 10;

3. размер ответа – 0.7KB;

4. время загрузки – 291ms.

Сделайте вывод, как параметры помогают уменьшить объём данных.

Параметры позволяют создать выборку необходимых данных, без необходимости запрашивать всё и сразу. При работе с большим объемом

данных это позволяет ускорить процесс получения данных, заставив сервер искать только необходимые по параметрам данные.

2.6. Массовые запросы

Откройте несколько URL одновременно:

- /posts/1
- /posts/2
- /posts/3

Зафиксируйте:

- как меняется время отклика;

/posts/1 – 785ms

/posts/2 – 780ms

/posts/3 – 775ms

- как браузер параллелит запросы;



Выполняет их одновременно, но время начала отправки запроса у всех разное

- нет ли избыточных запросов.

Нет

2.7. Заголовки и ответы

1. Во вкладке Headers выпишите не менее 5 ключевых заголовков, например:

2. Content-Type – не задан

3. Cache-Control - max-age=43200

4. ETag - W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"

5. Date - Thu, 15 Oct 2026 11:15:42 GMT

6. Server - cloudflare

7. Сравните вкладки Preview и Response:

8. чем они отличаются по представлению данных;

В Preview данные почему то показываются снизу-вверх, в Response, где представлен json наоборот, оригинальный вариант так скажем

9. в каких случаях удобнее использовать Preview, а в каких – Response.

Preview	Ответ, отрендеренный браузером: JSON сворачивается в дерево, HTML рендерится, картинки показываются как картинки, шрифты как шрифты
Response	Сырой текст ответа ровно в том виде, в каком пришёл по сети

Если нам необходимо увидеть сырые данные, которые идут от сервера, чтобы выяснить, что отправлено, то лучше response

Preview лучше использовать, что бы увидеть конечный, отрендеренный вариант полученного ответа, тобиж если будет получена картинка, то в response будет показан её исходный код, а в preview сама картинка и так со всеми данными.

2.8. Оптимизация запросов

Сравните ответы:

Запрос	Размер ответа	Время загрузки
/posts	0.7KB	219ms
/posts?userId=1	0.6KB	648ms
/posts/1	0.9KB	785ms

Предложите меры оптимизации:

– Предлагаю использовать пагинацию (ограничение количества записей на страницу);

– Предлагаю использовать фильтрацию (по userId, статусам и т.п.);

– Предлагаю использовать уменьшение размера ответа (убрать лишние поля);

– Предлагаю использовать кэширование на клиенте и сервере;

– Предлагаю использование правильных заголовков (Cache-Control, ETag и т.д.).

2.9. Работа с API через Postman

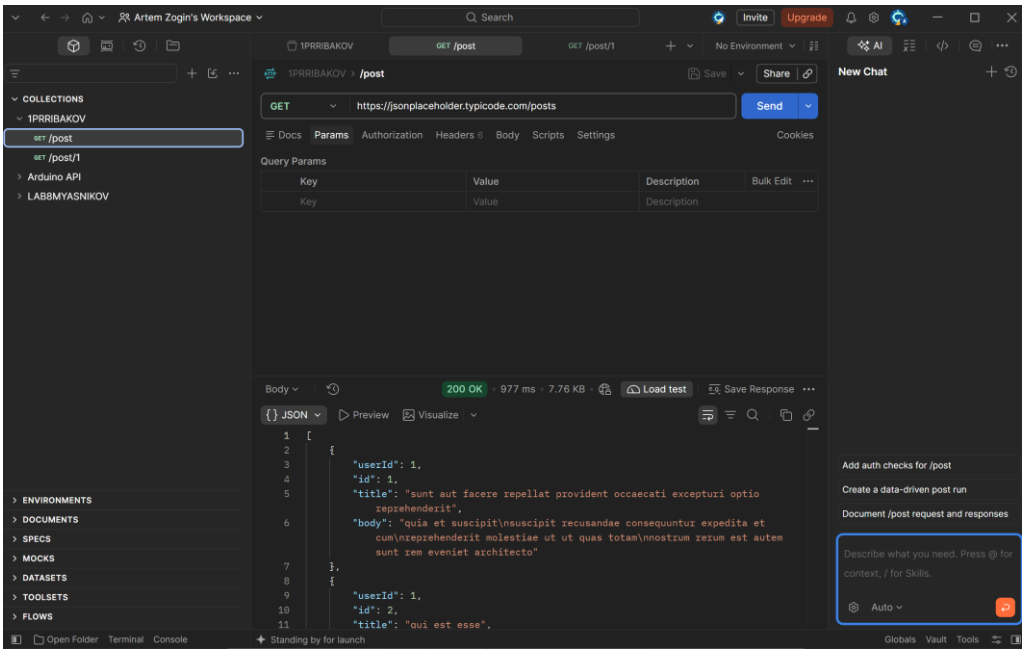
1. Перейдите на сайт:

https://www.postman.com/

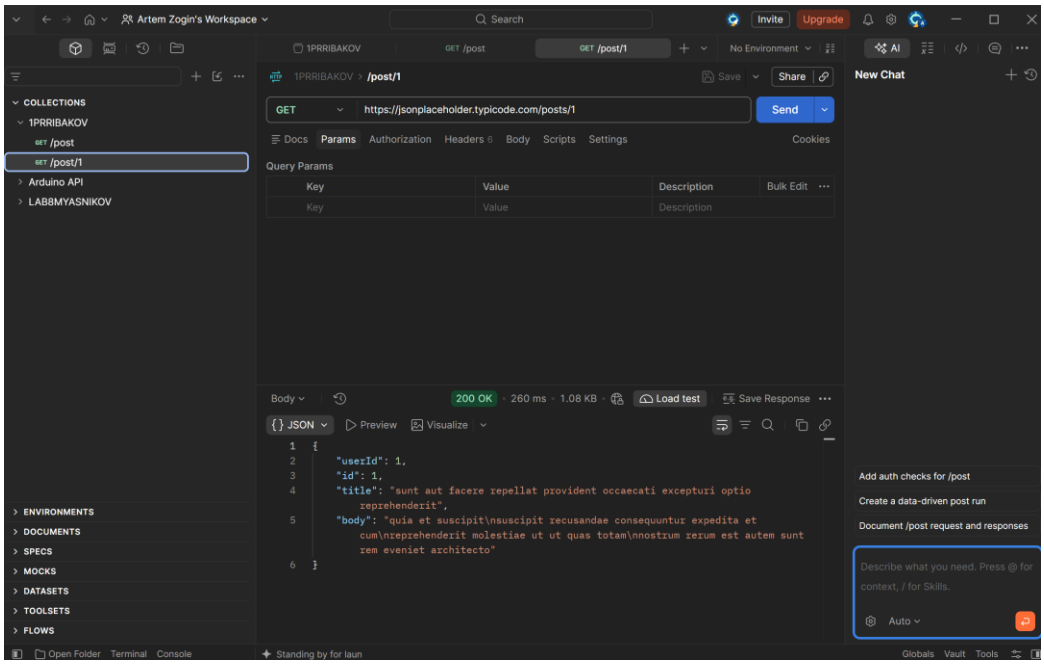
2. Создайте новый workspace.

3. Выполните:

1. GET /posts



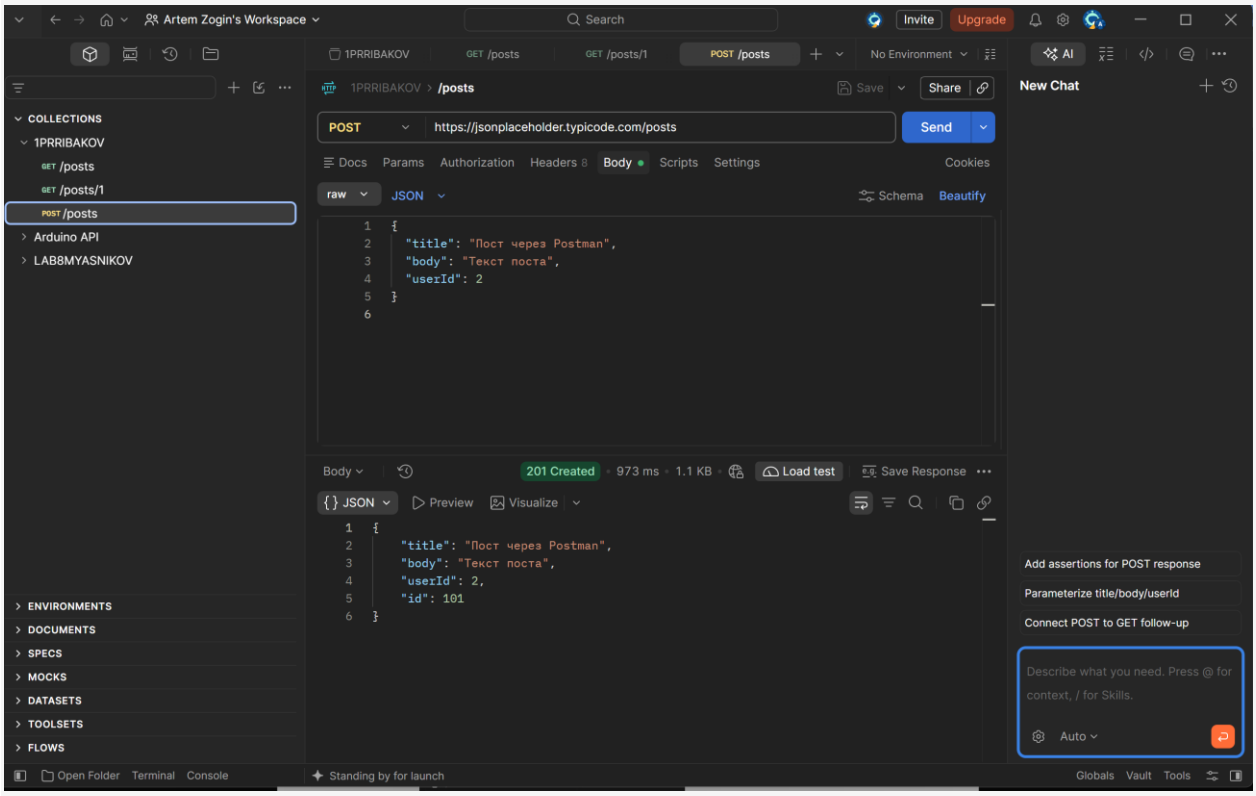
2. GET /posts/1



Сравните ответы. Сравнил

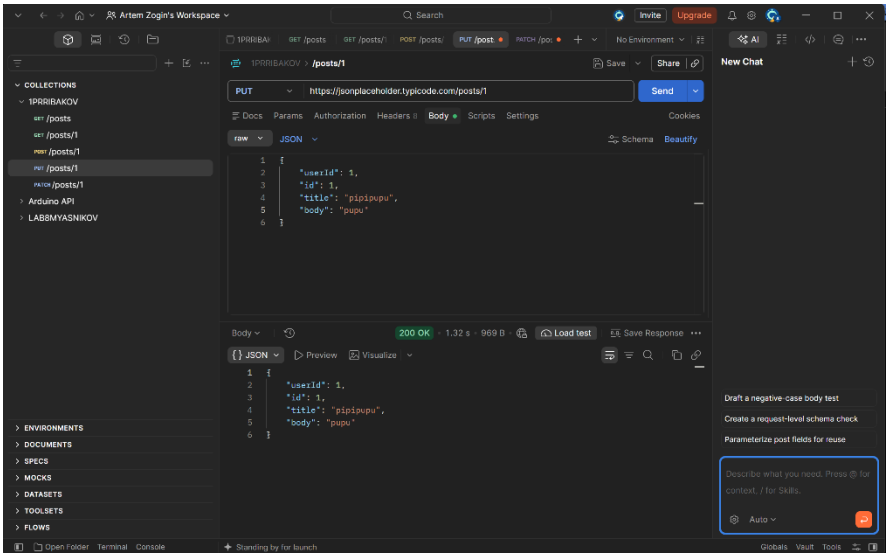
1. Выполните POST /posts с JSON-телом:

```
{
  "title": "Пост через Postman",
  "body": "Текст поста",
  "userId": 2
}
```

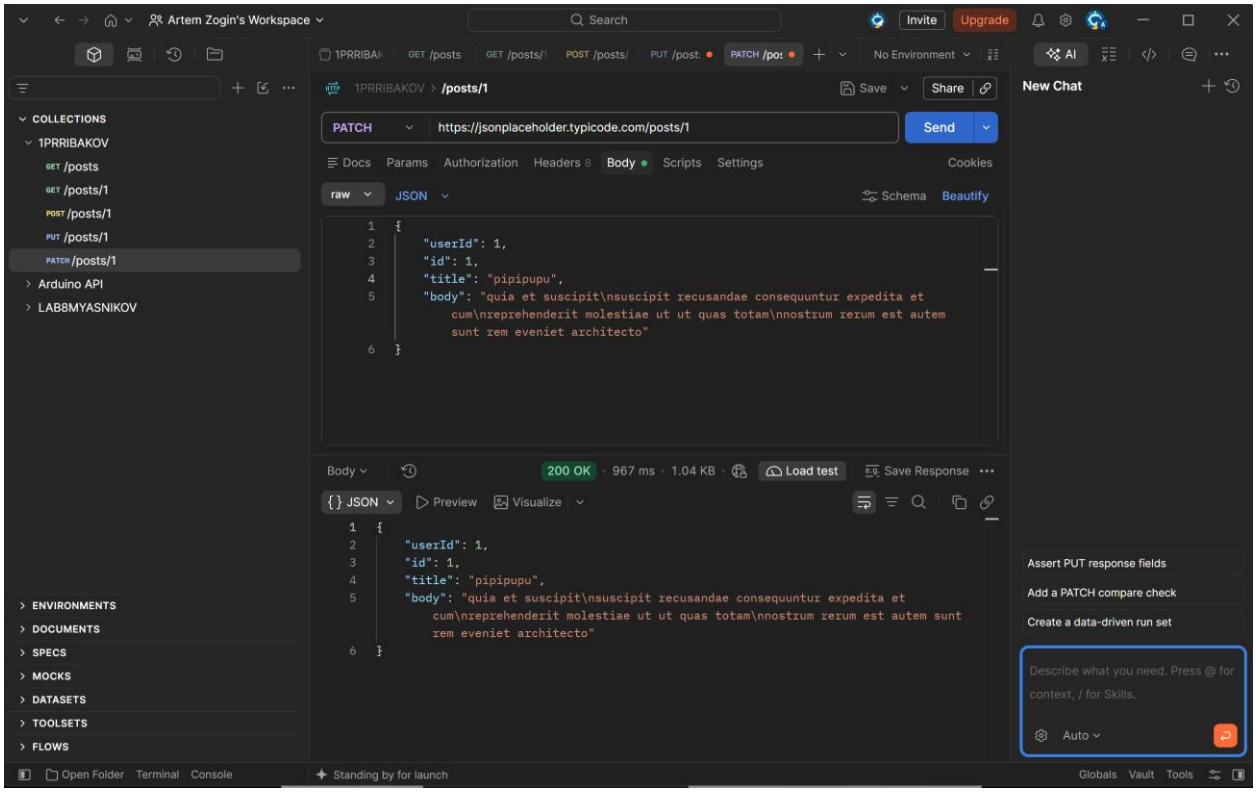


1. Выполните:

2. PUT /posts/1 – полное обновление ресурса;



3. PATCH /posts/1 – частичное обновление (изменить только один-два поля).



Объясните, чем отличаются результаты.

GET /posts – ПОЛУЧЕНИЕ ВСЕХ ЗАПИСЕЙ

GET /posts/1 –ПОЛУЧЕНИЕ ЗАПИСИ С ID = 1

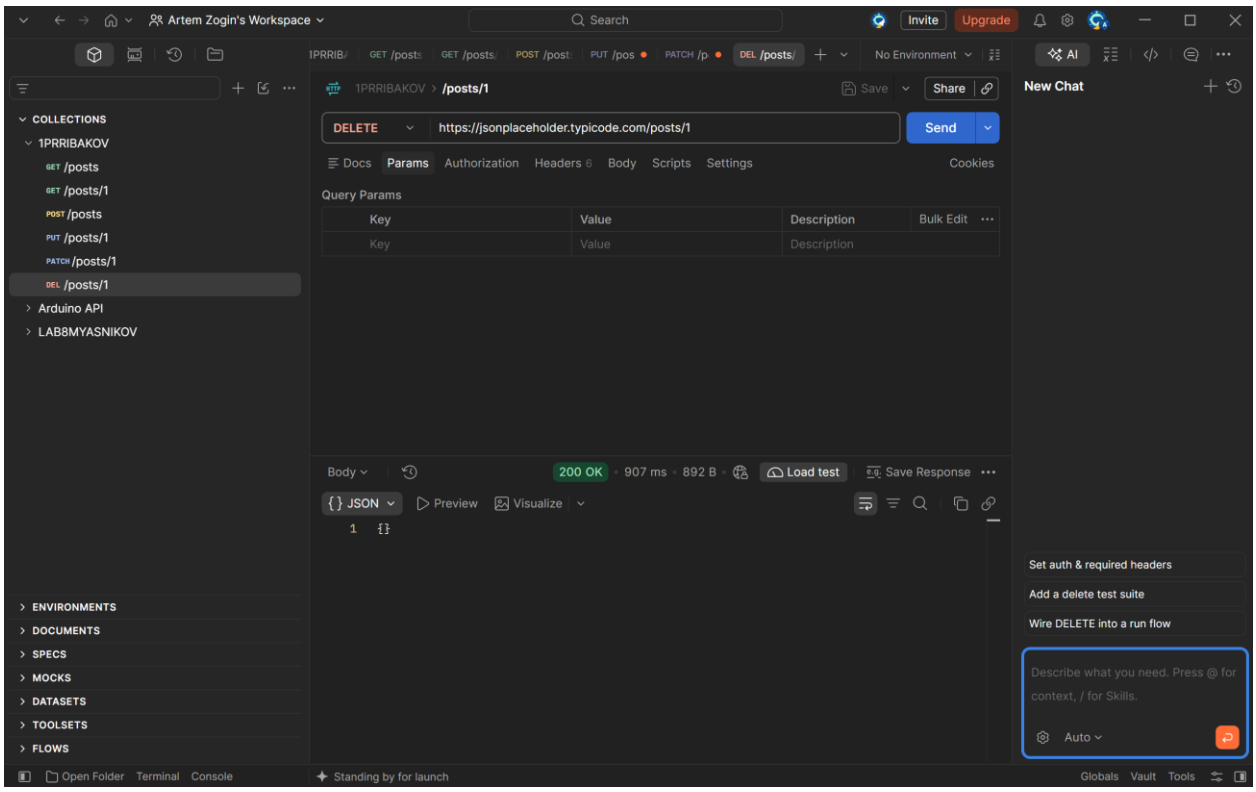
POST /posts – ДОБАВЛЕНИЕ НОВОЙ ЗАПИСИ К ОБЩЕЙ МАССЕ

PUT /posts/1 – ИЗМЕНЕНИЕ ЗАПИСИ С ID=1 ЦЕЛИКОМ

PATCH /posts/1 – ИЗМЕНЕНИЕ ЗАПИСИ С ID=1

Разница между PUT и PATCH в том, что в первом случае нам необходимо в запросе указать все свойства объекта, отчего величина запроса растёт, а PATCH только необходимые к изменению свойства, отчего размер запроса может кратно уменьшиться

1. Выполните DELETE /posts/1 и зафиксируйте:



2. код ответа – 200;

3. тело ответа (если есть).

```
1  {}
```

2.10. Сравнение DevTools и Postman

Критерий	DevTools	Postman
Запросы	Реальные – то, что шлёт страница	Ручные – что составили сами
Анализ запросов страницы	Все XHR/fetch, статика, инициатор	Нет
Производительность	Waterfall, TTFB, очередь, Connection ID	Только общее время ответа
Кэширование	200/304, from cache, Cache-Control	Не эмулируется
Контекст сайта	Cookies, сессия, CORS – как у пользователя	Изолированная среда
Ручное конструирование	Только повтор уже сделанного запроса	Полный контроль
Сохранение коллекций	Нет	Коллекции, папки, экспорт
Тесты и скрипты	Нет	Pre-request Script, Tests, Newman
Авторизация и токены	Видно, как реально уходит токен	Environments, {{token}}, OAuth
Автоматизация	Нет	Collection Runner, CI/CD

DevTools – когда нужно увидеть, что реально делает страница: отладка фронтенда, производительность, кэш, поиск избыточных запросов.

Postman – когда нужно самому сконструировать, сохранить, протестировать и автоматизировать запросы к API

3. Контрольные вопросы

1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?

Ответ:

DevTools (Developer Tools) – встроенный в браузер набор инструментов для разработки и отладки веб-страниц.

– Elements – просмотр и редактирование DOM и CSS в реальном времени: можно менять HTML, стили, смотреть вычисленные значения, box model, применённые правила.

– Console – вывод ошибок и логов JS, выполнение произвольного JS-кода, отладка выражений.

– Network – мониторинг всех сетевых запросов страницы: URL, метод, статус, заголовки, тело, время, размер, инициатор.

2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

Ответ:

Открыть: F12, Ctrl+Shift+I (Windows/Linux), Cmd+Option+I (macOS), либо правой кнопкой «Исследовать элемент».

Важные настройки Network:

– Disable cache – отключает кэш, чтобы видеть реальные запросы к серверу.

– Throttling – эмуляция медленной сети (Slow 3G, Fast 3G) для реалистичных замеров.

– Preserve log – сохранять запросы при перезагрузке/переходах.

– Колонки: Time, TTFB (Waiting), Content Download, Size, Waterfall, Connection ID, Protocol.

– Фильтры: Fetch/XHR, Doc, CSS, JS, Img – чтобы не утонуть в статике.

– User Agent – эмуляция другого браузера/устройства.

3. Перечислите основные части HTTP-запроса (метод, URL/параметры, заголовки, тело) и их назначение.

Ответ:

Часть	Назначение
Метод	GET, POST, PUT, PATCH, DELETE
URL и параметры	Path – путь query (?key=value) – параметры фильтрации/сортировки/пагинации
Заголовки (Headers)	Content-Type, Authorization, Accept, Cookie, User-Agent и др.
Тело (Body)	Данные запроса (обычно у POST/PUT/PATCH): JSON, form-data, файлы
Версия HTTP	HTTP/1.1, HTTP/2, HTTP/3 – влияет на производительность

4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE? Приведите по одному практическому примеру.

Ответ:

Метод	Назначение	Пример
GET	Получить ресурс	GET /posts/1 – читаем пост
POST	Создать ресурс	POST /posts с телом {title, body} – создать пост
PUT	Полностью заменить ресурс	PUT /posts/1 с полным объектом – перезаписать пост
PATCH	Частично изменить ресурс	PATCH /posts/1 с {title: "new"} – поменять только заголовок
DELETE	Удалить ресурс	DELETE /posts/1 – удалить пост

5. Для чего используются методы HEAD и OPTIONS и где встречается preflight-запрос?

Ответ:

– HEAD – как GET, но сервер возвращает только заголовки без тела. Используется, чтобы проверить: существует ли ресурс, его размер, Content-Type, Last-Modified – не скачивая тело.

– OPTIONS – узнать, какие методы и заголовки поддерживает сервер для ресурса. Возвращает Allow: GET, POST, ...

– Preflight-запрос – автоматический OPTIONS, который браузер отправляет перед «сложным» CORS-запросом (нестандартный метод, кастомные заголовки, Content-Type: application/json). Сервер должен ответить Access-Control-Allow-* заголовками, иначе основной запрос не уйдёт.

6. Расскажите о группах кодов ответов (1xx–5xx) и приведите примеры для 200, 301/302, 304, 404, 500.

Ответ:

Группа	Смысл	Примеры
1xx	Информационные	100 Continue, 101 Switching Protocols
2xx	Успех	200 OK – запрос выполнен; 201 Created – ресурс создан
3xx	Перенаправление	301 Moved Permanently – постоянный редирект; 302 Found – временный; 304 Not Modified – из кэша
4xx	Ошибка клиента	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Ошибка сервера	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Примеры из задания:

200 – /posts/1 успешно вернул JSON.

304 – браузер отправил If-None-Match, сервер ответил «не изменилось», тело не передаётся – экономия трафика.

7. Что такое REST и как ресурс в REST представлен на уровне URL?

Ответ:

REST (Representational State Transfer) – архитектурный стиль для API, где:

– всё – ресурсы, к которым обращаются по URL;

– действия выражаются HTTP-методами (GET, POST, PUT, PATCH, DELETE);

– обмен обычно в JSON;

– запросы stateless – сервер не хранит состояние клиента между запросами.

Ресурс в URL – объект или объекты:

– /posts – коллекция постов

– /posts/1 – конкретный пост

8. Какие типы данных поддерживает JSON? Приведите короткий пример JSON-объекта и массива.

Ответ:

Типы: string, number, boolean, null, object, array.

```
{
  "id": 1,
  "title": "Hello",
  "published": true,
  "views": 42,
  "tags": ["js", "api"],
  "author": {
    "name": "Ivan",
    "email": null
  }
}
```

9. В чём отличие вкладок Preview и Response в карточке запроса DevTools?

Ответ:

Preview – ответ, отрендеренный браузером, показывается как страница, картинки.

Response – сырой текст ответа ровно как пришёл. Удобно копировать, диффить, отлаживать парсинг, проверять кодировку и что реально вернул сервер

10. Какие заголовки чаще всего анализируются при оптимизации (например, Content-Type, Cache-Control, ETag, Authorization)?

Ответ:

– Content-Type – что за данные (application/json, text/html, image/webp).

– Content-Encoding – сжатие (gzip, br) – уменьшает трафик.

– Cache-Control – правила кэширования (max-age, no-cache, no-store).

– ETag / If-None-Match – валидация кэша, ответ 304.

– Authorization – токен доступа (Bearer, Basic).

– Accept / Accept-Encoding – что клиент готов принять.

– Content-Length – размер тела.

11. Какие приёмы снижения нагрузки на сеть можно использовать для API: пагинация, фильтрация, кэширование и др.?

Ответ:

– Пагинация – не отдавать всё сразу.

– Фильтрация и сортировка – меньше данных.

– Выбор полей – не тянуть лишнее.

– Кэширование

– Сжатие – gzip/brotli

12. Чем работа с API в Postman отличается от анализа запросов в DevTools, и когда уместен каждый инструмент?

Ответ:

DevTools: когда нужна отладка фронтенда, анализ реальных запросов страницы, производительность, кэширование, поиск избыточных запросов, CORS-проблемы.

Postman: когда необходима разработка и тестирование API, сохранение и переиспользование запросов, автотесты, работа с токенами и окружениями, шаринг с командой.